

Progetto di Laboratorio di Basi di Dati

Indice

1. Analisi dei Requisiti	2
1.1. Glossario	2
1.2. Strutturazione dei Requisiti	2
1.3. Operazioni	3
2. Progettazione Concettuale	4
2.1. Vincoli di Integrità	4
3. Progettazione Logica	6
3.1. Ristrutturazione del Modello E-R	6
3.1.1. Tavola dei Volumi	6
3.1.2. Analisi delle Ridondanze	6
3.1.2.1. Ridondanza 1: Attributo derivato «Numero di Film Prodotti»	6
3.1.3. Eliminazione delle Generalizzazioni	7
3.1.4. Traduzione degli Attributi Multivalore	7
3.1.5. Eliminazione della Relazione Quaternaria	7
3.1.6. Schema E-R Ristrutturato	8
3.1.7. Scelta degli Identificatori Primari	8
3.2. Traduzione nello Schema Relazionale	8
3.2.1. Vincoli di Integrità	9
4. Progettazione Fisica	12
4.1. Creazione dello Schema in SQL DDL	12
5. Implementazione	16
5.1. Creazione della Base di Dati	16
5.2. Creazione delle Tabelle	16
5.3. Implementazione dei Trigger	16
5.3.1. Trigger 1: Una persona deve essere o un attore, o un regista, o entrambi	16
5.3.2. Trigger 2: Ci può essere al massimo un noleggio attivo	17
5.3.3. Trigger 3: Attributo derivato «Numero di Film Prodotti»	17
5.3.4. Trigger 4: Intervalli di noleggio non sovrapposti	17
5.4. Popolamento della Base di Dati	18
5.5. Interrogazioni	18
5.5.1. Interrogazione 1	19
5.5.2. Interrogazione 2	19
5.5.3. Interrogazione 3	19
5.5.4. Interrogazione 4	20
5.5.5. Interrogazione 5	20
5.5.6. Interrogazione 6	20

1. Analisi dei Requisiti

Si vuole progettare una base di dati per la gestione di informazioni sull'industria cinematografica. La consegna segue dall'esercizio 3 del compito di basi di dati del 22 giugno 2009, con alcune aggiunte.

1.1. Glossario

È stato prodotto un glossario dei termini che compaiono nel testo, insieme ad eventuali sinonimi e collegamenti ad altri termini individuati.

Termine	Descrizione	Sinonimi	Collegamenti
Film			Attori, Azienda Produttrice, Registi, Frase Significative
Attore	Recita in uno o più film	Autore	Film
Regista	Dirige almeno un film, e può recitare in uno o più film		Film
Copia fisica di un Film			Film, Cliente
Azienda Produttrice	Produce uno o più film		Film
Cliente	Noleggia una o più copie fisiche di film	Cliente Registrato	Film

Tabella 1: Glossario

1.2. Strutturazione dei Requisiti

Frase di carattere generale
Si supponga di aver collezionato il seguente insieme di requisiti per la progettazione di una base di dati relazionale per la gestione di informazioni sull'industria cinematografica.

Frase relative a film
• Ogni film sia identificato univocamente dal suo titolo e dall'anno di produzione (assumiamo che in uno stesso anno non possano venir prodotti due o più film con lo stesso titolo, ma ammettiamo che film con lo stesso titolo possano essere prodotti in anni diversi, come accade nel caso dei remake). • Ogni film abbia una durata espressa in minuti, un'unica azienda produttrice e sia classificato come appartenente ad uno o più generi (commedia, thriller, film dell'orrore, fantasy, ..) • Ogni film abbia uno o più registi e zero, uno o più autori che vi recitano. • Ogni film sia caratterizzato da una breve descrizione della trama. • Ogni film abbia zero o più frasi significative, ciascuna pronunciata da uno degli attori che recitano nel film (assumiamo che alcuni attori possano pronunciare più frasi significative, altri una sola frase significativa, altri ancora nessuna).

Frasi relative a attori
• Gli attori siano identificati univocamente dal nome e dalla data di nascita (assumiamo che non vi siano attori con lo stesso nome nati lo stesso giorno). • Ogni attore compaia in almeno un film. • Ogni attore svolga uno o più ruoli in ogni film nel quale recita.
Frasi relative a registi
• I registi siano identificati univocamente dal nome e dalla data di nascita (assumiamo che non vi siano registi con lo stesso nome nati lo stesso giorno). • Ogni regista diriga almeno un film. • Un regista possa anche recitare in uno o più film, inclusi film da lui/lei diretti.
Frasi relative a aziende produttrici
• Le aziende produttrici siano identificate dal loro nome e abbiano un unico recapito. • Ogni azienda produttrice produca uno o più film.
Frasi relative a videonoleggio (aggiunta alla consegna)
Si vuole gestire il videonoleggio di film da parte dei clienti registrati. • Un cliente può noleggiare una o più copie fisiche di un film per un certo periodo di tempo limitato. Se una copia fisica risulta prestata, non può essere rinoleggiata prima che essa venga restituita. • Si vuole ricordare lo storico dei prestiti dei film da parte dei clienti.

1.3. Operazioni

Le Operazioni 1-4 sono state definite per poter preparare delle interrogazioni, sviluppate in Sezione 5.5, mentre le Operazioni 5 e 6 con le relative frequenze per l'analisi dei costi e delle ridondanze, sviluppate in Sezione 3.1.2.

- *Operazione 1:* Ottieni il numero medio di attori che hanno partecipato in film di uno specifico genere.
- *Operazione 2:* Ottieni le coppie di clienti registrati che hanno visto gli stessi film.
- *Operazione 3:* Ottieni il regista che ha diretto il numero massimo di film.
- *Operazione 4:* Ottieni tutti gli attori che hanno recitato solo a film della stessa casa produttrice.
- *Operazione 5:* Inserimento di nuovo film prodotto da una data casa produttrice. Frequenza: 57 inserimenti al giorno¹
- *Operazione 6:* Calcola il numero di film prodotti da una data casa produttrice. Frequenza: 50 richieste al giorno²

¹Dal sito web IMDB, risulta che nell'anno 2024 sono stati rilasciati 20844 film, ovvero circa 57 film al giorno.

²Valore ipotetico - media rispetto a tutte le case produttrici.

2. Progettazione Concettuale

Come strategia di progettazione è stata scelta la strategia *inside out*, partendo dall'entità Film e man mano allargando lo schema al resto dei concetti identificati. Per questo è risultato utile il Glossario, da cui si è dedotto come Film fosse l'entità principale da cui tutte le altre dipendessero.

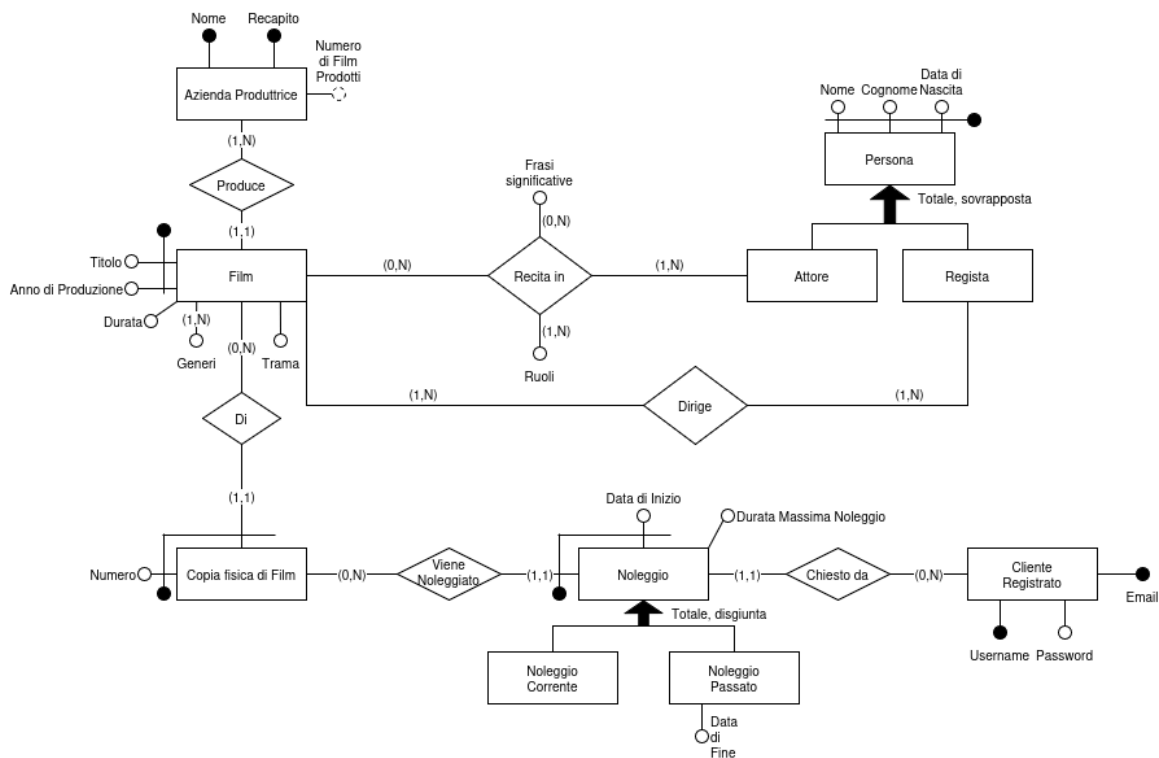


Figura 1: Schema Entità-Relazioni

Alcune note:

- Per **Copia fisica di Film**, entità debole di **Film**, si è usato il pattern di progettazione *istanza di*. Una copia fisica ha un numero che la identifica tra tutte le copie fisiche di un determinato film.
- Per **Azienda Produttrice** viene mantenuto il numero di film da essa prodotti come un attributo derivato (derivato dalla relazione **Produce**).
- Per i noleggi si è usato il pattern per la storicizzazione, quindi specializzando il **Noleggio** in **Noleggio Corrente** e **Noleggio Passato**; quest'ultimo ha l'attributo **Data di Fine**.
- Per **Cliente Registrato** sono presenti più chiavi candidate.
- Per **Azienda Produttrice** sono presenti più chiavi candidate.

2.1. Vincoli di Integrità

I seguenti vincoli di integrità devono essere aggiunti al precedente schema per garantire il significato atteso della base di dati.

- Una **Copia fisica** di un **Film** noleggiata non può essere rinoleggiata prima che venga restituita.
- La **Data di Inizio** del **Noleggio** deve essere antecedente alla **Data di Fine Noleggio**.

Nota:

- Il ciclo “Regista - Dirige - Film - Recita in - Attore” non è problematico, perché un Attore può Recitare in un Film che dirige.

- La durata massima del noleggio può essere maggiore della differenza tra la data di fine noleggio e la data di inizio noleggio, nel caso in cui la restituzione della copia fisica del film avvenga in ritardo.

3. Progettazione Logica

3.1. Ristrutturazione del Modello E-R

3.1.1. Tavola dei Volumi

Concetto	Tipo	Volume
Film	Entità	730000 ³
Produce	Relazione	730000

3.1.2. Analisi delle Ridondanze

Assumiamo un costo di un accesso in lettura di 1, e in scrittura di 3.

3.1.2.1. Ridondanza 1: Attributo derivato «Numero di Film Prodotti»

Presenza di ridondanza			
Operazione 5			
Concetto	Costrutto	Accessi	Tipo
Film	Entità	1	S
Azienda Produttrice	Entità	1	L
Azienda Produttrice	Entità	1	S
Produce	Relazione	1	S
$\begin{aligned}\text{Totale} &= (3 * \text{CostoLettura} + 1 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (3 * 1 + 1 * 3) * 57 \\ &= 342 \text{ unità di costo al giorno}\end{aligned}$			
Operazione 6			
Concetto	Costrutto	Accessi	Tipo
Azienda Produttrice	Entità	1	L
$\begin{aligned}\text{Totale} &= (1 * \text{CostoLettura} + 0 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (1 * 1 + 0 * 3) * 50 \\ &= 50 \text{ unità di costo al giorno}\end{aligned}$			
Totale = 342 + 50 = 392 unità di costo al giorno			

³Il sito web IMDB contiene 731089 film alla data di scrittura.

Assenza di ridondanza			
Operazione 5			
Concetto	Costrutto	Accessi	Tipo
Film	Entità	1	S
Produce	Relazione	1	S
$\begin{aligned} \text{Totale} &= (1 * \text{CostoLettura} + 1 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (1 * 1 + 1 * 3) * 57 \\ &= 228 \text{ unità di costo al giorno} \end{aligned}$			
Operazione 6			
Concetto	Costrutto	Accessi	Tipo
Produce	Relazione	730000	L
$\begin{aligned} \text{Totale} &= (730000 * \text{CostoLettura} + 0 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (730000 * 1 + 0 * 3) * 50 \\ &= 36500000 \text{ unità di costo al giorno} \end{aligned}$			
Totale = 228 + 36500000 = 36500228 unità di costo al giorno			
Scegliamo di mantenere la ridondanza.			

3.1.3. Eliminazione delle Generalizzazioni

- Generalizzazione Persona: Siccome la generalizzazione è totale e sovrapposta, usiamo il metodo ibrido per la rimozione della generalizzazione, in cui aggiungiamo per ogni figlio una relazione con il genitore. Le entità figli diventano entità deboli. Aggiungiamo un vincolo di integrità per fare in modo che Persona si specializzi obbligatoriamente in almeno uno tra Attore e Resista.
- Generalizzazione Noleggio: Siccome la generalizzazione è totale e disgiunta, applichiamo la tecnica della rimozione dei figli. Per discriminare tra noleggio corrente e noleggio passato, utilizziamo l'attributo opzionale «Data di fine» come discriminante.

3.1.4. Traduzione degli Attributi Multivalore

Traduzione degli attributi «Frase significative» e «Ruoli» della relazione «Recita in».

- «Frase Significative»: aggiungiamo un'entità debole, in relazione (0, N) con «Recita in», dato che una stessa frase significativa può essere detta da attori diversi o in film diversi.
- «Ruolo»: aggiungiamo un'entità debole, in relazione (1, 1) con «Recita in».

Traduzione dell'attributo multivalore «Generi»: aggiungiamo una entità (non debole) in relazione (1, N) con «Film».

3.1.5. Eliminazione della Relazione Quaternaria

Siccome la relazione «Recita in», dopo la traduzione degli attributi multivalore, risulta quaternaria, è stata reificata in una nuova entità «Recitazione», debole rispetto a «Film» e «Attore».

3.1.6. Schema E-R Ristrutturato

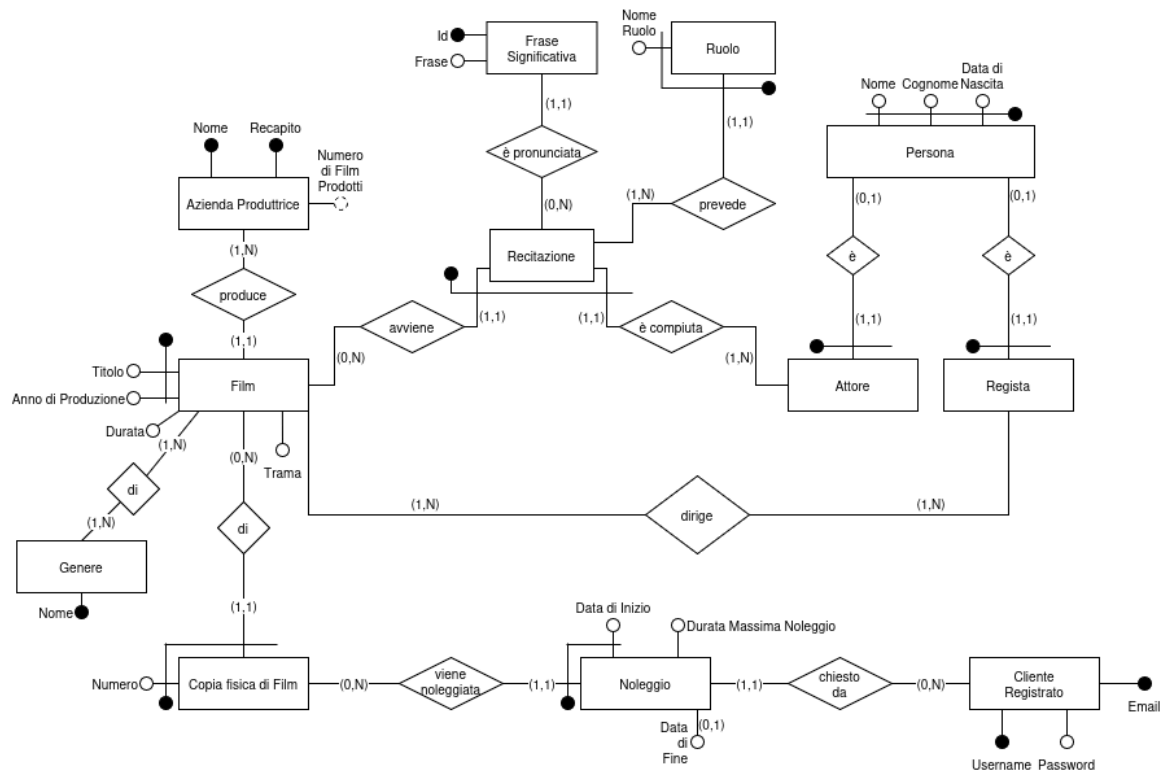


Figura 2: Schema Entità-Relazioni Ristrutturato

Vincoli di integrità:

- Una copia fisica di un film noleggiata non può essere rinoleggiata prima che venga restituita.
- Una persona deve essere o un Attore, o un Regista, o entrambi.
- La data di inizio del noleggio deve essere antecedente alla data di fine noleggio.

Nota:

- Siccome **Frase Significativa** è identificata da una complessa chiave composta contenente anche una stringa possibilmente molto lunga, è giustificabile usare una chiave surrogata per identificarla, sebbene non sia teoricamente richiesto.

3.1.7. Scelta degli Identificatori Primari

- Per «Cliente Registrato» abbiamo due chiavi candidate: username e email. Si sceglie username.

3.2. Traduzione nello Schema Relazionale

AziendaProduttrice (Nome, Recapito, NumeroDiFilmProdotti)

VNN: {Recapito, NumeroDiFilmProdotti}

UNIQUE: {Recapito}

Film (Titolo, AnnoDiProduzione, Durata, Trama, AziendaProduttrice)

FK: {AziendaProduttrice} → {AziendaProduttrice.Nome}

VNN: {Durata, Trama, AziendaProduttrice}

Genere (Nome)

GenereDelFilm (TitoloFilm, AnnoDiProduzioneFilm, NomeGenere)

FK: {TitoloFilm, AnnoDiProduzioneFilm} → {Film.Titolo, Film.AnnoDiProduzione}

FK: {NomeGenere} → {Genere.Nome}

CopiaFisicaDiFilm (Numero, TitoloFilm, AnnoFilm)

FK: {TitoloFilm, AnnoFilm} → {Film.Titolo, Film.AnnoDiProduzione}

Noleggio (DataDiInizio, NumeroCopia, TitoloFilm, AnnoFilm, EmailCliente, DurataMassimaNoleggio, DataDiFine)

FK: {NumeroCopia, TitoloFilm, AnnoFilm} → {CopiaFisicaDiFilm.Numero, CopiaFisicaDiFilm.TitoloFilm, CopiaFisicaDiFilm.AnnoFilm}

FK: {EmailCliente} → {ClienteRegistrato.Email}

VNN: {EmailCliente, DurataMassimaNoleggio}

ClienteRegistrato (Email, Username, Password)

UNIQUE: {Username}

VNN: {Password, Username}

Persona (Nome, Cognome, DataDiNascita)

Attore (Nome, Cognome, DataDiNascita)

FK: {Nome, Cognome, DataDiNascita} → {Persona.Nome, Persona.Cognome, Persona.DataDiNascita}

Regista (Nome, Cognome, DataDiNascita)

FK: {Nome, Cognome, DataDiNascita} → {Persona.Nome, Persona.Cognome, Persona.DataDiNascita}

RegistaDelFilm (TitoloFilm, AnnoDiProduzioneFilm, NomeRegista, CognomeRegista, AnnoDiNascitaRegista)

FK: {TitoloFilm, AnnoDiProduzioneFilm, NomeRegista, CognomeRegista, AnnoDiNascitaRegista} → {Film.Titolo, Film.AnnoDiProduzione, Regista.Nome, Regista.Cognome, Regista.AnnoDiNascita}

Ruolo (NomeRuolo, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)

FK: {TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore} → {Recitazione.TitoloFilm, Recitazione.AnnoFilm, Recitazione.NomeAttore, Recitazione.CognomeAttore, Recitazione.DataNascitaAttore}

FraseSignificativa (ID, Frase, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)

FK: {TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore} → {Recitazione.TitoloFilm, Recitazione.AnnoFilm, Recitazione.NomeAttore, Recitazione.CognomeAttore, Recitazione.DataNascitaAttore}

VNN: {Frase}

Recitazione (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)

FK: {TitoloFilm, AnnoFilm} → {Film.Titolo, Film.AnnoDiProduzione}

FK: {NomeAttore, CognomeAttore, DataNascitaAttore} → {Attore.Nome, Attore.Cognome, Attore.DataDiNascita}

3.2.1. Vincoli di Integrità

La traduzione dallo schema concettuale allo schema logico (schema relazionale) rende necessaria l'aggiunta dei seguenti vincoli di integrità causati dalla perdita di espressività del modello relazionale, oltre ai vincoli già evidenziati inizialmente.

I vincoli intra-relazionali, escludendo i vincoli di chiave primaria, di unicità, e di NOT NULL che sono stati già individuati in Sezione 3.2, sono i seguenti:

- Noleggio: DataDiFine > DataDiInizio if DataDiFine IS NOT NULL
- Film: Durata > 0

I vincoli inter-relazionali, escludendo i vincoli di chiave esterna che sono stati già individuati in Sezione 3.2, sono i seguenti; per ogni vincolo sono riportate le azioni che potrebbero violare l'integrità della base di dati:

- Un'azienda produttrice deve aver prodotto almeno un film.
 - Inserimento di Azienda Produttrice
 - Cancellazione di Film
 - Modifica dell'attributo «AziendaProduttrice» di Film
- Un genere deve essere associato ad almeno un film
 - Inserimento Film
 - Cancellazione Film
 - Modifica di «Genere» nella relazione «GenereDelFilm»
 - Cancellazione di «Genere» nella relazione «GenereDelFilm»
- Un film deve essere associato ad almeno un genere
 - Cancellazione di Genere
- Un Film deve avere almeno un Regista
 - Inserimento di Film
 - Cancellazione di Regista
 - Modifica di «RegistaDelFilm»
 - Cancellazione di «RegistaDelFilm»
- Una Recitazione deve prevedere almeno un Ruolo
 - Cancellazione di Ruolo
 - Inserimento di Recitazione
- Un Attore deve recitare in almeno un Film
 - Inserimento di Attore
 - Cancellazione di Recitazione
- Un Regista deve dirigere almeno un Film
 - Inserimento Regista
 - Cancellazione di Film
 - Modifica di «RegistaDelFilm»
 - Cancellazione di «RegistaDelFilm»
- Una persona deve essere o un attore, o un regista, o entrambi:
 - Inserimento di Persona
 - Cancellazione di Attore
 - Cancellazione di Regista
- Ci può essere al massimo un noleggio attivo
 - Inserimento Noleggio
 - Modifica Noleggio
- Gli intervalli generati dalle date di inizio e di fine noleggio, per ogni copia fisica di un film, non devono sovrapporsi
 - Inserimento Noleggio
 - Modifica Noleggio
- Attributo derivato «Numero di Film Prodotti»
 - Inserimento Film

- Modifica Film (attributo AziendaProduttrice)
- Cancellazione Film

Notiamo che, per gli quanto riguardano gli intervalli generati dalle date relative ai noleggi, si ipotizza di lavorare con intervalli chiusi. Segue, quindi, che se un noleggio termina il giorno 10, un nuovo noleggio per la stessa copia fisica di un film può iniziare a partire dal giorno 11.

Questi vincoli di integrità devono essere implementati utilizzando dei meccanismi esterni, dato che questi non possono essere garantiti nel modello relazionale. Per i vincoli intra-relazionali verranno usati dei controlli **CHECK**, mentre per i vincoli inter-relazionali dei *trigger* o le clausole **ON DELETE** e **ON UPDATE** dei vincoli di chiave esterna.

4. Progettazione Fisica

4.1. Creazione dello Schema in SQL DDL

Viene riportato il codice di creazione delle relazioni definite in Sezione 3.2 nel linguaggio Data Definition Language (DDL) di SQL.

```
CREATE TABLE AziendaProduttrice (  
    Nome VARCHAR(255) PRIMARY KEY,  
    Recapito VARCHAR(255) UNIQUE NOT NULL,  
    NumeroDiFilmProdotti INT NOT NULL  
);  
  
CREATE TABLE Genere (  
    Nome VARCHAR(100) PRIMARY KEY  
);  
  
CREATE TABLE ClienteRegistrato (  
    Email VARCHAR(255) PRIMARY KEY,  
    Username VARCHAR(100) UNIQUE NOT NULL,  
    Password VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE Persona (  
    Nome VARCHAR(100),  
    Cognome VARCHAR(100),  
    DataDiNascita DATE,  
  
    PRIMARY KEY (Nome, Cognome, DataDiNascita)  
);  
  
CREATE TABLE Attore (  
    Nome VARCHAR(100),  
    Cognome VARCHAR(100),  
    DataDiNascita DATE,  
  
    PRIMARY KEY (Nome, Cognome, DataDiNascita),  
    FOREIGN KEY (Nome, Cognome, DataDiNascita)  
        REFERENCES Persona(Nome, Cognome, DataDiNascita)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE  
);  
  
CREATE TABLE Regista (  
    Nome VARCHAR(100),  
    Cognome VARCHAR(100),  
    DataDiNascita DATE,  
  
    PRIMARY KEY (Nome, Cognome, DataDiNascita),  
    FOREIGN KEY (Nome, Cognome, DataDiNascita)  
        REFERENCES Persona(Nome, Cognome, DataDiNascita)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE  
);  
  
CREATE TABLE Film (  

```

```

    Titolo VARCHAR(255),
    AnnoDiProduzione INT,
    Durata INT NOT NULL CHECK (Durata > 0),
    Trama TEXT NOT NULL,
    AziendaProduttrice VARCHAR(255) NOT NULL,

    PRIMARY KEY (Titolo, AnnoDiProduzione),
    FOREIGN KEY (AziendaProduttrice)
        REFERENCES AziendaProduttrice(Nome)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE RegistaDelFilm (
    TitoloFilm VARCHAR(255),
    AnnoDiProduzioneFilm INT,
    NomeRegista VARCHAR(100),
    CognomeRegista VARCHAR(100),
    DataDiNascitaRegista DATE,

    PRIMARY KEY (TitoloFilm, AnnoDiProduzioneFilm, NomeRegista, CognomeRegista,
DataDiNascitaRegista),
    FOREIGN KEY (TitoloFilm, AnnoDiProduzioneFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (NomeRegista, CognomeRegista, DataDiNascitaRegista)
        REFERENCES Regista(Nome, Cognome, DataDiNascita)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE GenereDelFilm (
    TitoloFilm VARCHAR(255),
    AnnoDiProduzioneFilm INT,
    NomeGenere VARCHAR(100),

    PRIMARY KEY (TitoloFilm, AnnoDiProduzioneFilm, NomeGenere),
    FOREIGN KEY (TitoloFilm, AnnoDiProduzioneFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (NomeGenere)
        REFERENCES Genere(Nome)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE CopiaFisicaDiFilm (
    Numero INT,
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,

    PRIMARY KEY (Numero, TitoloFilm, AnnoFilm),
    FOREIGN KEY (TitoloFilm, AnnoFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)

```

```

        ON UPDATE CASCADE
        ON DELETE CASCADE
    );

CREATE TABLE Recitazione (
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,
    NomeAttore VARCHAR(100),
    CognomeAttore VARCHAR(100),
    DataNascitaAttore DATE,

    PRIMARY KEY (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore),
    FOREIGN KEY (TitoloFilm, AnnoFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (NomeAttore, CognomeAttore, DataNascitaAttore)
        REFERENCES Attore(Nome, Cognome, DataDiNascita)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE Ruolo (
    NomeRuolo VARCHAR(100),
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,
    NomeAttore VARCHAR(100),
    CognomeAttore VARCHAR(100),
    DataNascitaAttore DATE,

    PRIMARY KEY (NomeRuolo, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore),
    FOREIGN KEY (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)
        REFERENCES Recitazione(TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE FraseSignificativa (
    ID INT,
    TitoloFilm VARCHAR(255) NOT NULL,
    AnnoFilm INT NOT NULL,
    NomeAttore VARCHAR(100) NOT NULL,
    CognomeAttore VARCHAR(100) NOT NULL,
    DataNascitaAttore DATE NOT NULL,
    Frase TEXT NOT NULL,

    PRIMARY KEY (ID),
    FOREIGN KEY (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)
        REFERENCES Recitazione(TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

```

```

CREATE TABLE Noleggio (
    DataDiInizio DATE,
    NumeroCopia INT,
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,
    EmailCliente VARCHAR(255) NOT NULL,
    DurataMassimaNoleggio INT NOT NULL,
    DataDiFine DATE CHECK (DataDiFine IS NULL OR DataDiFine > DataDiInizio),

    PRIMARY KEY (DataDiInizio, NumeroCopia, TitoloFilm, AnnoFilm),
    FOREIGN KEY (NumeroCopia, TitoloFilm, AnnoFilm)
        REFERENCES CopiaFisicaDiFilm(Numero, TitoloFilm, AnnoFilm)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (EmailCliente)
        REFERENCES ClienteRegistrato(Email)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

```

Codice 1: Creazione dello Schema

5. Implementazione

5.1. Creazione della Base di Dati

Per creare una nuova base di dati, eseguire il seguente comando da shell:

```
psql -c "CREATE DATABASE industria_cinematografica;"
```

5.2. Creazione delle Tabelle

Per eseguire il codice SQL DDL di Codice 1, eseguire sulla base di dati appena creata:

```
psql -d industria_cinematografica -c "\i setup/create.sql"
```

5.3. Implementazione dei Trigger

Vengono presentate le implementazioni dei seguenti trigger, scelti per le diverse tipologie di operazioni necessarie per mantenere i vincoli di integrità:

1. *Una persona deve essere o un attore, o un regista, o entrambi*: Trigger in inserimento su Persona.
2. *Ci può essere al massimo un noleggio attivo*: Trigger in modifica su Noleggio.
3. *Attributo derivato «Numero di Film Prodotti»*: Trigger in cancellazione su Film.
4. *Intervalli di noleggio non sovrapposti*: Trigger in inserimento su Noleggio.

Per il setup dei triggers, eseguire:

```
psql -d industria_cinematografica -c "\i setup/trigger_1.sql"
psql -d industria_cinematografica -c "\i setup/trigger_2.sql"
psql -d industria_cinematografica -c "\i setup/trigger_3.sql"
psql -d industria_cinematografica -c "\i setup/trigger_4.sql"
```

5.3.1. Trigger 1: Una persona deve essere o un attore, o un regista, o entrambi

```
CREATE FUNCTION ControllaSpecializzazione() RETURNS trigger AS $$
BEGIN
    IF NOT EXISTS (
        SELECT *
        FROM Attore
        WHERE Nome = NEW.Nome AND Cognome = NEW.Cognome AND DataDiNascita =
NEW.DataDiNascita
    ) AND NOT EXISTS (
        SELECT *
        FROM Regista
        WHERE Nome = NEW.Nome AND Cognome = NEW.Cognome AND DataDiNascita =
NEW.DataDiNascita
    ) THEN
        RAISE EXCEPTION 'La persona deve essere o un attore, o un regista, o entrambi';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER Persona_ControllaSpecializzazioneInInserimento
AFTER INSERT ON Persona
DEFERRABLE INITIALLY DEFERRED
```

```
FOR EACH ROW
EXECUTE FUNCTION ControllaSpecializzazione();
```

5.3.2. Trigger 2: Ci può essere al massimo un noleggio attivo

```
CREATE FUNCTION ControllaNoleggioAttivo() RETURNS trigger AS $$
BEGIN
  IF EXISTS (
    SELECT *
    FROM Noleggio
    WHERE
      TitoloFilm = NEW.TitoloFilm AND
      AnnoFilm = NEW.AnnoFilm AND
      NumeroCopia = NEW.NumeroCopia AND
      DataDiInizio <> NEW.DataDiInizio AND
      DataDiFine IS NULL
  ) THEN
    RAISE EXCEPTION 'Ci può essere al massimo un noleggio attivo per questa copia
fisica di film';
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE CONSTRAINT TRIGGER Noleggio_ControllaNoleggioAttivoInModifica
AFTER UPDATE ON Noleggio
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION ControllaNoleggioAttivo();
```

5.3.3. Trigger 3: Attributo derivato «Numero di Film Prodotti»

```
CREATE FUNCTION AggiornaNumeroFilmProdotti() RETURNS trigger AS $$
BEGIN
  UPDATE AziendaProduttrice
  SET NumeroDiFilmProdotti = (
    SELECT COUNT(*)
    FROM Film
    WHERE AziendaProduttrice = OLD.AziendaProduttrice
  )
  WHERE Nome = OLD.AziendaProduttrice;
  RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER Film_AggiornaNumeroFilmProdottiInRimozione
AFTER DELETE ON Film
FOR EACH ROW
EXECUTE FUNCTION AggiornaNumeroFilmProdotti();
```

5.3.4. Trigger 4: Intervalli di noleggio non sovrapposti

```
CREATE OR REPLACE FUNCTION ControllaIntervalliNoleggio()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$
```

```

DECLARE
    CNT INTEGER;
BEGIN
    -- Seleziona i noleggi che sono in overlap
    SELECT COUNT(*) INTO CNT
    FROM   Noleggio N
    WHERE  N.NumeroCopia = NEW.NumeroCopia
          AND N.TitoloFilm = NEW.TitoloFilm
          AND N.AnnoFilm   = NEW.AnnoFilm
          AND (NEW.DataDiFine IS NULL OR N.DataDiInizio <= NEW.DataDiFine)
          AND (N.DataDiFine IS NULL OR NEW.DataDiInizio <= N.DataDiFine);

    IF CNT > 0
    THEN
        RAISE EXCEPTION 'Il Noleggio è invalido.';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_ControllaIntervalliNoleggio
BEFORE INSERT ON Noleggio
FOR EACH ROW
EXECUTE FUNCTION ControllaIntervalliNoleggio();

```

5.4. Popolamento della Base di Dati

Per il popolamento della base di dati ci si è affidati ad uno script Python. Per la generazione di un dataset realistico, si è utilizzata la libreria *faker*, capace di generare dei dati realistici di vari domini, come nomi di aziende, di persone, ecc. L'uso del linguaggio di programmazione Python insieme a *faker* ci ha permesso di popolare la base di dati con dati piuttosto veritieri e con volumi compatibili con quelli supposti in Sezione 3.1.1.

L'ordine degli inserimenti deve rispettare i vincoli definiti. In particolare, sono state evidenziate queste dipendenze cicliche tra relazioni, causate dalla partecipazione obbligatoria delle relazioni coinvolte. L'inserimento deve dunque avvenire in una transazione con il controllo dei vincoli posticipato (DEFERRED) al COMMIT.

- Persona ↔ Attore/Regista: una Persona deve essere o un Attore o un Regista o entrambi; un Attore/Regista deve essere una Persona
- Azienda ↔ Film: un'Azienda deve avere prodotto almeno un Film; un Film deve essere prodotto da un'Azienda
- Film ↔ Genere: un Film deve avere un Genere; un Genere deve essere associato ad un Film
- Film ↔ Regista: un Film deve avere un Regista che lo dirige; il Regista deve dirigere un Film
- Attore ↔ Recitazione ↔ Ruolo: un Attore deve partecipare ad almeno una Recitazione; la Recitazione si riferisce a un Attore e richiede almeno un Ruolo.

5.5. Interrogazioni

Le interrogazioni proposte di seguito fanno riferimento alle operazioni definite in Sezione 1.3.

5.5.1. Interrogazione 1

-- Troviamo per ogni film il numero di attori che vi recitano. Notiamo che ogni istanza di recitazione corrisponde ad un ed un solo un attore. Non serve quindi effettuare join con attore

```
CREATE VIEW NumeroAttoriPerFilm AS
SELECT Film.Titolo AS TitoloFilm,
       Film.AnnoDiProduzione AS AnnoFilm,
       COUNT(Recitazione.*) AS NumeroAttori
FROM Film JOIN Recitazione ON
       Film.Titolo = Recitazione.TitoloFilm AND
       Film.AnnoDiProduzione = Recitazione.AnnoFilm
GROUP BY Film.Titolo, Film.AnnoDiProduzione;
```

-- Utilizziamo la vista precedente per ottenere il numero medio di attori per genere

```
SELECT GenereDelFilm.NomeGenere, AVG(NumeroAttori)
FROM GenereDelFilm
JOIN NumeroAttoriPerFilm ON
       GenereDelFilm.TitoloFilm = NumeroAttoriPerFilm.TitoloFilm AND
       GenereDelFilm.AnnoDiProduzioneFilm = NumeroAttoriPerFilm.AnnoFilm
GROUP BY GenereDelFilm.NomeGenere;
```

5.5.2. Interrogazione 2

```
SELECT C1.Email, C2.Email
FROM   ClienteRegistrato C1, ClienteRegistrato C2
WHERE  NOT EXISTS ( -- non esiste un Film che ha visto C1 ma non C2
        SELECT *
        FROM   Noleggio N1
        WHERE  N1.EmailCliente = C1.Email AND
        NOT EXISTS (
                SELECT *
                FROM   Noleggio N2
                WHERE  N2.EmailCliente = C2.Email    AND
                N1.TitoloFilm = N2.TitoloFilm AND
                N1.AnnoFilm = N2.AnnoFilm
            )
    )
    AND
    NOT EXISTS ( -- non esiste un Film che ha visto C2 ma non C1
        SELECT *
        FROM   Noleggio N1
        WHERE  N1.EmailCliente = C2.Email AND
        NOT EXISTS (
                SELECT *
                FROM   Noleggio N2
                WHERE  N2.EmailCliente = C1.Email    AND
                N1.TitoloFilm = N2.TitoloFilm AND
                N1.AnnoFilm = N2.AnnoFilm
            )
    )
    AND C1.Email < C2.Email;
```

5.5.3. Interrogazione 3

```
CREATE VIEW NumeroFilmPerRegista(NomeRegista, CognomeRegista, DataDiNascitaRegista,
NumeroFilm) AS
```

```

SELECT Regista.Nome, Regista.Cognome, Regista.DataDiNascita, COUNT(*)
FROM Regista
  JOIN Film ON Film.NomeRegista = Regista.Nome AND
             Film.CognomeRegista = Regista.Cognome AND
             Film.DataDiNascitaRegista = Regista.DataDiNascita
GROUP BY Regista.Nome, Regista.Cognome, Regista.DataDiNascita;

```

```

SELECT NomeRegista, CognomeRegista, DataDiNascitaRegista
FROM NumeroFilmPerRegista
WHERE
  NumeroFilm >= ALL (
    SELECT NumeroFilm
    FROM NumeroFilmPerRegista
  );

```

5.5.4. Interrogazione 4

```

CREATE VIEW Attore_AziendeProd AS
  SELECT NomeAttore, CognomeAttore, DataNascitaAttore, AziendaProduttrice
  FROM   Recitazione JOIN Film
        ON Recitazione.TitoloFilm = Film.Titolo
        AND Recitazione.AnnoFilm  = Film.AnnoDiProduzione;

```

```

SELECT A1.NomeAttore, A1.CognomeAttore, A1.DataNascitaAttore
FROM   Attore_AziendeProd A1
WHERE  NOT EXISTS (
  SELECT *
  FROM   Attore_AziendeProd A2
  WHERE  A1.NomeAttore      = A2.NomeAttore
        AND A1.CognomeAttore = A2.CognomeAttore
        AND A1.DataNascitaAttore = A2.DataNascitaAttore
        AND A1.AziendaProduttrice <> A2.AziendaProduttrice
)

```

```

-- Alternativa
-- SELECT *
-- FROM   Recitazione JOIN Film
--       ON Recitazione.TitoloFilm = Film.Titolo
--       AND Recitazione.AnnoFilm  = Film.AnnoDiProduzione
-- GROUP BY NomeAttore, CognomeAttore, DataNascitaAttore
-- HAVING  COUNT(DISTINCT AziendaProduttrice) = 1;

```

5.5.5. Interrogazione 5

```

INSERT INTO Film (Titolo, AnnoDiProduzione, Durata, Trama, AziendaProduttrice,
NomeRegista, CognomeRegista, DataDiNascitaRegista)
VALUES ('Titolo del film', 2024, 120, 'Trama del film', 'Nome casa produttrice',
'Mario', 'Rossi', '1970-01-01');

```

5.5.6. Interrogazione 6

```

SELECT Nome, NumeroFilmProdotti
FROM AziendaProduttrice;

```